

Snakemake workshop

A small workflow for Vienna heat days

Today in 90 minutes

Part	Time	Outcome
Input and demo	25 min	Read a rule, request a file, inspect the DAG
Independent work	45 min	Build, observe, and generalize a workflow
Wrap-up	20 min	Explain reruns and identify useful next steps

By the end, you can turn a short script chain into a workflow you can request, inspect, and extend.

Working scripts are only the beginning

```
bash scripts/download_data.sh
for nc in raw/*.nc; do
    ncap2 -O -s 'tas=tas+273.15' "$nc" "intermediate/${nc##*/}"
done
python scripts/analyze_data.py
python scripts/plot_data.py
```

The commands may be correct. The analysis structure still lives partly in memory, notes, shell history, and directory conventions.

What a workflow makes explicit

Manual execution leaves me asking...	A declared workflow records...
Which command produced this file?	matching inputs and outputs
What is stale after a change?	affected jobs in the DAG [*]
Which work can run independently?	independent branches
What counts as complete?	concrete targets

The shift is from writing down execution order to declaring how results are produced.

^{*} DAG = Directed Acyclic Graph

Our small case: a file contract

```
rule prepare_temperature_data:
    input:
        raw="data/raw/vienna_hohe_warte_tlmax.csv"
    output:
        clean="data/processed/vienna_hohe_warte_tlmax.csv"
    shell:
        "python scripts/prepare_data.py "
        "--input {input.raw} --output {output.clean}"
```

The paths declare what is required and produced. The command fulfills that contract.

Ask for a concrete file

```
snakemake -n -p data/processed/vienna_hohe_warte_tlmax.csv
```

- The final argument is the target file.
- `-n` describes the planned work without executing it.
- `-p` prints the rendered shell command.
- `{input.raw}` and `{output.clean}` become the named paths from the rule.

The target determines which part of the workflow is relevant.

File contracts form the DAG



For a requested file, Snakemake works backwards: find a matching output, collect its inputs, and continue until every input exists or can be produced.

A target rule defines “complete”

```
rule all:  
    input:  
        "figures/heat_days_ge_30.png"  
    default_target: True
```

Now this is enough:

```
snakemake -n -p
```

`all` is a convention. `default_target: True` explicitly selects the rule used when no target is given.

A rule is a pattern; a job is concrete

```
output:  
  counts="results/heat_days_ge_{threshold}.csv"
```

Requested file:

```
results/heat_days_ge_30.csv → threshold = 30
```

One wildcard rule can create many concrete jobs without copying the rule.

`expand()` constructs concrete targets

```
THRESHOLDS = [25, 30, 35]

rule all:
    input:
        expand(
            "figures/heat_days_ge_{threshold}.png",
            threshold=THRESHOLDS,
        )
    default_target: True
```

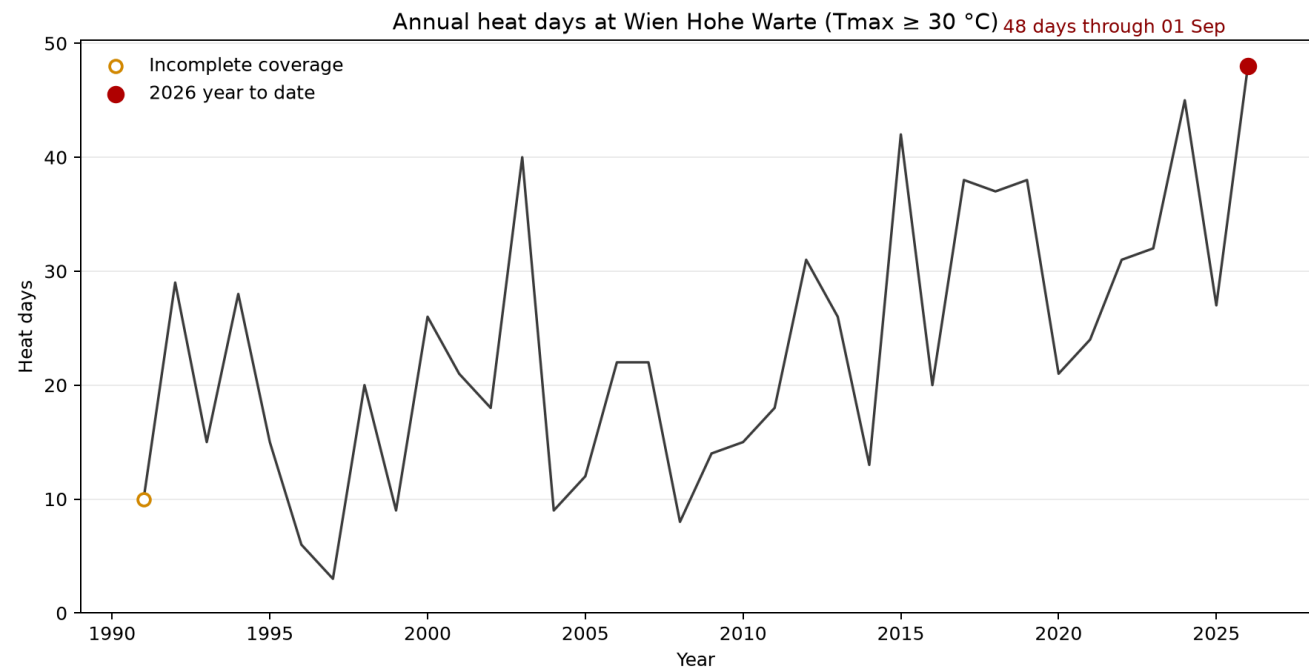
The producing rules contain wildcard patterns. `expand()` returns the three specific filenames that define the complete result.

Request, inspect, run

```
snakemake -s solutions/Snakefile -n -p \  
figures/heat_days_ge_30.png
```

```
snakemake -s solutions/Snakefile -j 1 \  
figures/heat_days_ge_30.png
```

```
snakemake -s solutions/Snakefile -j 1 \  
figures/heat_days_ge_30.png
```



Source: GeoSphere Austria, klima-v2-1d

Reruns follow the requested result

Workflow state	Expected work
All files current	Nothing to do
Figure missing	Plot only
Count and figure missing	Count, then plot
All generated files missing	Prepare, count, then plot

A dry-run makes these decisions inspectable before execution.

Independent work — 45 minutes

1. Build the concrete 30 °C count-and-plot chain.
2. Observe its no-op and partial-rebuild behaviour.
3. Generalize it to 25, 30, and 35 °C.

Use the handout `exercises/workshop.md`. The complete solution is a recovery path if the hints do not unblock you.

Debug from the target backwards

When Snakemake surprises you, ask:

1. What reason does the dry-run give?
2. What exact file did I request?
3. Which rule output can produce it?
4. Which input files does that rule require?

Start with paths and targets before inspecting the Python scripts.

When asked by Snakemake, add `--unlock` or `--rerun-incomplete` flags.

Wrap-up — share your experience and needs

- What helped you getting started?
- With what did you struggle?
- What support would you likely need for Snakemake-adoption?
- Which additional functionality would you need?

Recap — The compact mental model

```
declare file contracts → request a result → inspect the DAG → execute  
→ repeat safely
```

Snakemake infers execution order from matching input and output files. Start with one useful target; generalize only when repeated outputs reveal a pattern.

Extend when a concrete need appears

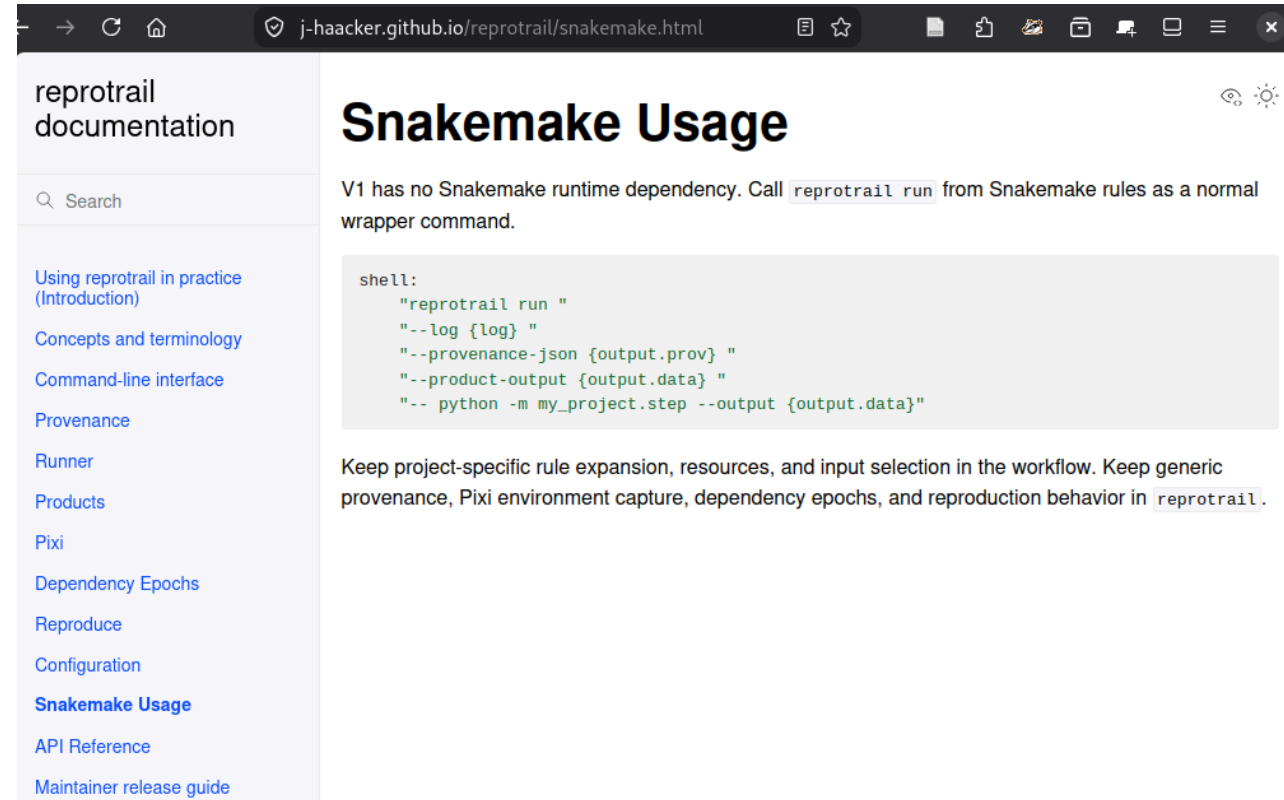
When...	Extend the workflow with...
Analysis values change	Configuration
Software must travel	Per-rule environments
Compute grows	Resources and an execution profile

These extend the same rule contracts; they do not require redesigning the workflow first.

A personal outlook: Reprotrail

Snakemake supplies an executable workflow. For this result, I am experimenting with preserving:

- the data snapshot and code revision;
- the resolved environment;
- the Snakemake run and resulting file.



The screenshot shows a web browser displaying the 'reprotrail documentation' website. The left sidebar contains a navigation menu with the following items: 'Using reprotrail in practice (Introduction)', 'Concepts and terminology', 'Command-line interface', 'Provenance', 'Runner', 'Products', 'Pixi', 'Dependency Epochs', 'Reproduce', 'Configuration', 'Snakemake Usage' (which is highlighted), 'API Reference', and 'Maintainer release guide'. The main content area is titled 'Snakemake Usage' and contains the following text: 'V1 has no Snakemake runtime dependency. Call `reprotrail run` from Snakemake rules as a normal wrapper command.' Below this text is a code block showing a shell command:

```
shell:
"reprotrail run "
"--log {log} "
"--provenance-json {output.prov} "
"--product-output {output.data} "
"-- python -m my_project.step --output {output.data}"
```

 At the bottom of the main content area, there is a paragraph: 'Keep project-specific rule expansion, resources, and input selection in the workflow. Keep generic provenance, Pixi environment capture, dependency epochs, and reproduction behavior in `reprotrail`.'

Snakemake lets you repeat workflows effortlessly

Start simple, grow your workflows on demand.